ITA03-MOBILE COMPUTING

ASSIGNMENT-5

ASSIGNMENT 5 – CO5

CO5 Coverage: Six Characteristics of mobile app - connectivity, convenience, localization, reachability, security and personalization - Android Intents and Services - Synchronization and Replication of Mobile Data - Android & IOS multimedia framework (Core Video framework and Core Media framework) - Peer- to Peer Architecture for mobile application -Performance and Multithreading -Graphics and UI Performance**.**

ANDROID APPLICATION FEATURES & PERFORMANCE ENGINEERING

Course Outcome – CO5: Create mobile application design using diverse mobile operating system frameworks to ensure performance efficiency, security, usability, and scalability.

Nature: Individual

Duration: One Week Total Marks: 50

Assessment Components

PART A – Design, Analysis & Pseudocode / Architectural Reasoning: 25 Marks

PART B – Implementation, Instrumentation & Computational Investigation: 25 Marks

Submission Date:5/11/2026

PART A – DESIGN, ANALYSIS AND ARCHITECTURAL REASONING (25 Marks)

A1 – Constraint-Aware Shuttle Tracking and Campus Alert Architecture (7 Marks) A1.1 – Problem Model

Element	Description
Input	GPS location, shuttle ID, timestamp, network state, battery level, location permission, user login state and alert messages.
Output	Updated shuttle position, cached route information, campus notifications and delivery status.
Decision	Collect location now or later; store locally or upload; choose a background worker or foreground service; send, defer or cancel an alert.
Constraints	Location permission, battery level, Doze mode, network availability, user privacy, authentication and Android background limits.
Goal	Show useful and recent shuttle information, deliver urgent alerts safely and avoid unnecessary battery or data use.

A1.2 – Runtime State

The application maintains a state record containing shuttle ID, last location, timestamp, route ID, task priority, upload status, retry count, permission status and authentication status. Global state stores battery percentage, network type, network quality, Doze status, logged-in user and maximum worker count.

A1.3 – Feasibility and Pruning Rules

- Do not collect continuous GPS updates when the user has disabled location permission.
- Defer routine location uploads when the network is unavailable or battery is below 15%.
- Allow an urgent campus safety alert to use an approved notification path, but still check authentication and privacy.
- Discard duplicate location events with the same shuttle ID and timestamp.
- Do not upload a location record after logout; retain or delete local data according to the application's privacy policy.
- Stop or postpone non-urgent background work during Doze mode.

A1.4 – Language-Independent Pseudocode

```
PROCEDURE PROCESS_CAMPUS_TASK(queue, deviceState)     WHILE queue is not
empty DO         task ← SELECT task with highest priority and oldest
creation time
        IF task.permissionRequired AND permission is invalid THEN
CANCEL task
            RECORD "permission denied"
            CONTINUE
        END IF

        IF task.requiresLogin AND userSession is invalid THEN
            CANCEL task
            RECORD "session expired"
            CONTINUE
        END IF

        IF task.requiresNetwork AND network is unavailable THEN
            DEFER task
            SAVE task for a later retry
            CONTINUE
        END IF

        IF battery < task.minimumBattery AND task.priority is not URGENT THEN
            DEFER task
            CONTINUE
        END IF

        IF deviceState is DOZE AND task.priority is not URGENT THEN
            SCHEDULE task for an allowed execution window
            CONTINUE
        END IF
         result ← EXECUTE task using an appropriate Android
component

        IF result is SUCCESS THEN
            MARK task completed
            SAVE acknowledgement
        ELSE IF result is TEMPORARY_FAILURE THEN
            INCREASE retry count
            SCHEDULE retry with backoff
        ELSE
            CANCEL task and record safe error
        END IF
    END WHILE
END PROCEDURE
```

A1.5 – Event Trace

Event	Constraint check	Action	Result

Shuttle starts moving	Location permission and battery valid	Collect GPS point and store it	Location saved
Mobile network is lost	Upload needs network	Keep latest point in local queue	No data lost
Battery falls to 12%	Routine upload threshold is 15%	Defer routine uploads	Battery use reduced
Campus emergency alert arrives	Message authenticated and notification permission available	Display urgent alert	Students informed
Student logs out	Session no longer valid	Cancel user-specific pending requests	No stale-session upload

Conclusion: The architecture separates location collection, local storage, upload and notification. Every task is checked before execution, and deferred tasks remain available for a later safe attempt.

A2 – Safe Synchronisation of Timetable and Attendance Data (5 Marks)

A2.1 – Problems in a Naïve Design

A design that uploads every local record wastes battery and network data. It may upload duplicate attendance entries, send private student information without encryption, ignore low battery and fail to resolve timetable conflicts. Repeating failed requests can also create duplicate records on the server.

A2.2 – Incremental Synchronisation Pseudocode

```
PROCEDURE SYNC_CAMPUS_RECORDS(records, deviceState)
    SORT records by priority and creation time

    FOR each record R in records DO
        IF R already acknowledged by server THEN
            DISCARD local duplicate
            CONTINUE
        END IF

        IF R contains personal data AND R is not encrypted THEN
            QUARANTINE R
            RECORD security error
            CONTINUE
        END IF

        IF user session is invalid THEN
            STOP synchronisation safely
        END IF

        IF network is unavailable OR battery is below safe threshold THEN
            DEFER R and keep local copy
            CONTINUE
        END IF

        IF server version differs from local base version THEN
            RESOLVE conflict using version number and update timestamp
            CONTINUE
        END IF

        UPLOAD R using a unique idempotency key

        IF server acknowledges R THEN
```

```
            MARK R as synchronised
        ELSE
            DEFER R with retry delay
        END IF
    END FOR
END PROCEDURE
```

A2.3 – Example Records and Pruning

Record	Priority / size / data	Condition	Decision
T1	High / 5 KB / timetable	Encrypted; Wi-Fi available	Upload
T2	Low / 80 KB / cached notice	Battery 9%; threshold 15%	Defer
T3	High / 2 KB / student attendance	Sensitive but not encrypted	Quarantine; stop upload branch
T4	Medium / 12 KB / route update	Network unavailable	Defer
T5	Low / 3 KB / old notice	Same record ID already acknowledged	Discard duplicate

Early pruning: T3 is removed from the upload path because sensitive attendance information is not encrypted. Retrying the upload cannot correct the security problem. The record must first be encrypted using an approved method.

A2.4 – Offline-First Challenge

If the application must support offline-first operation, temporary network failure must not cause a record to be deleted. The application should store changes durably, assign unique IDs and synchronise them later. A conflict should be retained until a defined merge rule can be applied. Therefore, pruning based only on temporary unavailability is not valid for permanent deletion; it is valid only for deferring work.

A3 – Common Pattern in Campus Background Tasks (5 Marks)

Design element	Shuttle location updates	Timetable synchronisation	Campus alerts
Component	Location service / scheduled worker	WorkManager worker and repository	Push messaging and notification manager
State	Last GPS point, permission, battery	Local version, server version, retry count	Message ID, delivery state, notification permission
Decision variable	Collect now or later	Upload, defer or merge	Display, group or suppress
Candidate choices	Collect, batch, pause	Upload, retry, defer, resolve conflict	Notify immediately, queue or suppress duplicate
Feasibility test	Location permission, battery and OS limits	Network, encryption, login and battery	Valid message, user settings and notification permission
Goal condition	Recent shuttle location available	Local and server records consistent	Important notice reaches intended students
Pruning / deferral	Skip redundant GPS samples	Wait for suitable network or battery	Suppress duplicate or expired notice
Undo / cancel	Stop location listener and release resources	Cancel worker but retain unsynced data	Cancel stale notification request

A3.1 – Common Pattern

All three tasks are state-driven background workflows. Each task selects work, checks constraints, executes or defers, records the result and handles cancellation. A common scheduler and state store can manage these tasks.

A3.2 – Generic Framework

```
PROCEDURE SCHEDULE_OR_EXECUTE(State)
```

```
    UPDATE State using operating-system and application events
item ← SELECT next eligible task

    IF no item is available THEN
        RETURN WAIT
    END IF

    IF permission, security or resource checks fail THEN
        DEFER or CANCEL item with a recorded reason
    ELSE
        SAVE current state
        EXECUTE item using a suitable worker or service

        IF execution succeeds THEN
            COMMIT result
        ELSE
            RESTORE safe state
            RETRY only if the failure is temporary and policy permits
        END IF
    END IF
END PROCEDURE
```

A3.3 – Importance of Undo and Cancellation

Undo and cancellation prevent incomplete tasks from leaving the application in an incorrect state. For example, if timetable synchronisation is cancelled during an upload, the local timetable must not be marked as synchronised until the server confirms receipt. Otherwise, a student may see an outdated timetable.

A4 – Local, Cloud and Hybrid Processing Choices (4 Marks)

Mobile problem	What is a solution?	Can it be verified?	Resource class and reason
QR-based classroom check-in	Validate a QR token and record attendance	Yes; compare accepted/rejected test scans and server records	Security-critical; attendance must be authenticated and protected
Cloud translation of lecture audio	Return translated text for permitted audio	Yes; measure translation accuracy and response time	Network-bound; depends on upload speed and cloud service
Nearby device file exchange	Transfer notes between nearby student devices	Yes; verify file hashes and transfer completion	Hybrid required; local transfer saves internet data but needs secure pairing
Encrypted student messaging	Only intended participants can read messages	Partly; protocol and security testing are needed	Security-critical; key handling and identity checks are essential
Live campus shuttle map	Display updated bus positions within an acceptable delay	Yes; measure location age, latency and map update rate	Hybrid required; local map rendering with server location updates
Offline campus navigation	Show routes from cached campus map data	Yes; compare route output and rendering time	On-device feasible; offline access is useful but map storage consumes memory

Why is 'runs slowly on a phone' not enough? A performance issue must be measured using a defined workload and metrics such as response time, frame time, memory, battery consumption, throughput and network delay. Slow behaviour may come from CPU work, storage, network conditions or contention.

Why does local/cloud placement matter? Local processing reduces network delay and can protect privacy, but uses device battery and memory. Cloud processing can use powerful servers but requires connectivity and sends data outside the device. Hybrid architecture chooses the best location for each task.

A5 – Adaptive Optimisation of the Campus Map Interface (4 Marks)

A5.1 – Exact Search

An exact method can test every combination of map image quality, tile-cache size and prefetch distance. If there are q quality levels, c cache sizes and p prefetch settings, it evaluates q × c × p configurations. As the number of settings grows, testing every combination takes more time and energy.

A5.2 – Adaptive Heuristic Pseudocode

```
PROCEDURE ADAPT_MAP(frameTime, memoryUse, battery, config)
    REPEAT
        MEASURE frameTime, memoryUse and battery
        IF frameTime > 16 ms OR memoryUse > memoryLimit THEN
            REDUCE map tile quality by one level
            REMOVE least-recently-used unused tiles
SHORTEN prefetch distance
        ELSE IF frameTime < 12 ms AND battery > 40%
                AND memoryUse has free space THEN
            INCREASE quality by one level OR extend prefetch distance
        END IF

        CHANGE only one major setting per measurement cycle
        RECORD the selected setting and reason
    UNTIL performance remains stable for several cycles
          OR no safe adjustment is available

    RETURN config
END PROCEDURE
```

A5.3 – Trade-off

The adaptive method can quickly maintain smooth map scrolling and reduce memory pressure. However, it does not guarantee the globally lowest energy use or the best image quality. It may choose a locally acceptable setting. This is reasonable for a live map because network and device conditions change frequently.

PART B – IMPLEMENTATION AND COMPUTATIONAL INVESTIGATION (25 Marks)

Note: The tables below contain illustrative simulation results to show how the investigation can be reported. Replace them with measurements from your own Android emulator or device before final submission.

B1 – Instrumented Background Architecture (7 Marks)

B1.1 – Test Scenarios

Test	Input condition	Expected behaviour	What it verifies
Normal operation	Battery 75%, Wi-Fi available, login and permissions valid	Eligible shuttle and timetable tasks complete	Successful execution
Low battery / Doze	Battery 10%, Doze active, routine task queued	Routine task is deferred and retained	Battery and background constraints
Permission revoked	Location permission removed while tracking is active	Location worker stops and no new GPS data is collected	Permission boundary and privacy

B1.2 – Design Compared with Implementation

The pseudocode describes a general scheduler. An Android implementation must persist task state because the operating system may stop the process. WorkManager can be used for deferrable uploads, while a foreground service should be reserved for user-visible ongoing work where platform rules allow it. Unique work IDs and server idempotency keys help prevent duplicate uploads.

B1.3 – FIFO and Priority Scheduling

Version 1 – FIFO: select the task that entered the queue first. Version 2 – Priority heuristic: calculate score = 4 × priority + 2 × age bonus + urgent flag. The task with the highest score is selected, but permission, encryption, network and battery checks are always applied first.

B1.4 – Illustrative Measurements

Measure	Version 1: FIFO	Version 2: Priority
Work items enqueued	12	12
Deferred / cancelled	5	5
Successful executions	7	7
Peak concurrent workers	2	2
Urgent alerts delivered in first group	1 of 3	3 of 3

Interpretation: Both approaches complete seven eligible tasks and defer five under the same constraints. Priority scheduling delivers urgent alerts earlier, although it does not reduce the total number of deferred tasks in this example.

B2 – Adversarial Workload: Same Size, Different Behaviour (5 Marks)

Property	Workload A – Easy	Workload B – Difficult
Number of records	10	10
Network	Stable Wi-Fi	Repeated weak signal and disconnects
Battery	65–90%	8–20%
Permission/session	Valid throughout	One permission revoked during execution
Record condition	Encrypted, mostly small	Some large records and one unencrypted sensitive record
Prediction	Most records complete on first attempt	More deferrals, retries and one security quarantine

Illustrative result	Workload A	Workload B
Items examined	10	10
Deferrals	1	5
Successful deliveries	9	4
Security quarantines	0	1
Retry attempts	1	4

Workload B requires more processing even though both workloads contain ten records. Its weak network causes repeated checks and retries, low battery causes deferrals, and the unencrypted record is quarantined. Input count alone does not describe energy or computational cost; the structure of the input and its constraints also matter.

B3 – Exact vs Approximate Map UI Optimisation (8 Marks)

B3.1 – Small Test Instances

For the exact method, evaluate all combinations and calculate cost = 3 × dropped-frame count + energy units + 2 × memory penalty. Reject any configuration that exceeds the frame-time or memory limit. The adaptive method starts with medium quality and changes settings based on measured performance.

Test instance	Exact minimum cost	Adaptive cost	Match?
Small map, stable device	14	14	Yes
Large tiles, low-memory phone	11	13	No
Medium map, strong device	9	9	Yes

These are illustrative simulation values, not physical device measurements. The second test shows that an adaptive heuristic may not find the exact minimum.

B3.2 – Counter-example

Assume the best configuration is high map quality with a small cache and cost 8. A medium-quality configuration with a larger cache has cost 10. The adaptive method observes a short frame-time spike, lowers quality and increases the cache. It settles at cost 10 and does not test the combination with cost 8. The local adjustment is reasonable but is not globally optimal.

B3.3 – Increasing Problem Size

Problem size	Exact configurations evaluated	Adaptive quality changes	Adaptive cache evictions
4 map items, 2 quality levels, 2 cache sizes	8	2	1
12 map items, 4 quality levels, 4 cache sizes	64	4	5
30 map items, 6 quality levels, 6 cache sizes	216	6	14

Exact search evaluates the complete configuration space. Adaptive search makes a limited number of changes based on current measurements. The operation counts are illustrative and should be replaced with values from the submitted program.

B3.4 – Engineering Trade-off

Approximate optimisation reduces decision time and responds to changing device conditions. It sacrifices the guarantee of finding the global minimum-cost configuration. It is suitable for live map interfaces because smooth interaction is more important than spending time testing every possible setting.

B4 – Multimedia Framework Engineering Decision (5 Marks)

The campus application allows students to record short event videos and upload them to a campus portal. Devices have different camera quality, battery levels, temperatures and network conditions.

B4.1 – Practical Encoding Heuristic

Selection rule: if battery is below 20% or the network is weak, choose 480p at 0.8 Mbps and defer upload until Wi-Fi or charging is available. If battery is at least 50%, temperature is normal and Wi-Fi is stable, choose 1080p at 3.5 Mbps. Otherwise choose 720p at 1.8 Mbps. Encrypt each clip and upload only after user authentication.

Device	Battery / network / temperature	Chosen setting	Upload decision	Illustrative estimate
A	85%, Wi-Fi, normal	1080p, 3.5 Mbps	Upload now	2 min; 3% battery

B	38%, 4G fair, normal	720p, 1.8 Mbps	Upload when stable	3 min; 4% battery
C	14%, weak 4G, normal	480p, 0.8 Mbps	Defer until WiFi/charging	1 min encode; 1% battery
D	60%, Wi-Fi, hot device	720p, 1.8 Mbps	Wait for temperature to fall	2 min; 2% battery

B4.2 – Challenge the Heuristic

A student may record an urgent safety incident while the battery is at 18% and Wi-Fi is excellent. The rule selects low resolution and defers upload, although the application may need to upload the clip quickly. This is a counter-example. A production system should include an explicit emergency priority option, user consent and a controlled battery policy.

B4.3 – Engineering Decision Note

Exhaustive search across resolution, bitrate, codec and upload timing is expensive when thousands of devices need decisions quickly.

The heuristic guarantees that its selected settings follow the stated rules, but it does not guarantee globally minimum energy or maximum video quality.

It reduces computation by checking a small number of conditions and can reduce energy through lower bitrate and deferred uploads.

Exact or cloud-assisted optimisation may be useful for offline planning or high-value media when sufficient time and connectivity are available.

Before deployment, monitor encoding time, upload latency, retries, battery drain, device temperature, video quality, security failures and completion rate.

Overall Conclusion

The smart campus example demonstrates how a mobile application can remain useful under changing network, battery and permission conditions. Constraint-aware scheduling prevents unnecessary background work, safe synchronisation protects student records, and reusable background patterns simplify implementation. Local, cloud and hybrid processing each have different benefits. Exact search is useful for small configuration spaces, while adaptive heuristics provide faster decisions for changing mobile conditions. Real deployment should be supported by device-based measurements and security testing.

Submission Note

For Part A, submit the problem models, language-independent pseudocode, event traces, tables and reasoning. For Part B, include your source code, test inputs, expected and actual outputs, instrumentation logs and interpretation. The numerical results in this document are examples only; run the program on an emulator or device and replace illustrative values with your own measured results.